

RFD: Document Q&A System with RAG-Powered Analysis

Project Name: Gemini RAG Bot - Document Q&A System **Version:** 2.0 **Last Updated:** January 2026 **Status:** Production

Table of Contents

- [1. Executive Summary](#)
 - [2. Project Overview](#)
 - [3. Architecture](#)
 - [4. Core Features](#)
 - [5. Database Schema](#)
 - [6. API & Edge Functions](#)
 - [7. Configuration System](#)
 - [8. Skills & Expert System](#)
 - [9. RAG Pipeline](#)
 - [10. User Interface](#)
 - [11. Technology Stack](#)
 - [12. Deployment](#)
 - [13. Future Roadmap](#)
-

1. Executive Summary

The Document Q&A System is a sophisticated RAG (Retrieval Augmented Generation) powered application that enables users to upload documents and ask questions about their content. The system leverages Google's Gemini AI for intelligent document analysis, featuring advanced chunking strategies, configurable retrieval techniques, and a flexible expert skills system.

Key Capabilities

- **Multi-format Document Support:** PDF, DOCX, TXT, JSON
 - **Advanced RAG Pipeline:** HyDE, query decomposition, verification, confidence scoring
 - **Expert Skills System:** Pre-built and AI-generated domain experts
 - **Configurable Retrieval:** Fusion search, reranking, self-RAG, CRAG
 - **Professional Output:** Audit-quality reports with citations
 - **Second Brain Features:** Skill Creator, Generator Skills, Tool Connectors
-

2. Project Overview

2.1 Problem Statement

Organizations need to efficiently extract insights from large document collections. Traditional keyword search fails to understand context and nuance. Manual review is time-consuming and inconsistent.

2.2 Solution

A RAG-powered document analysis system that:

- Intelligently chunks and indexes documents

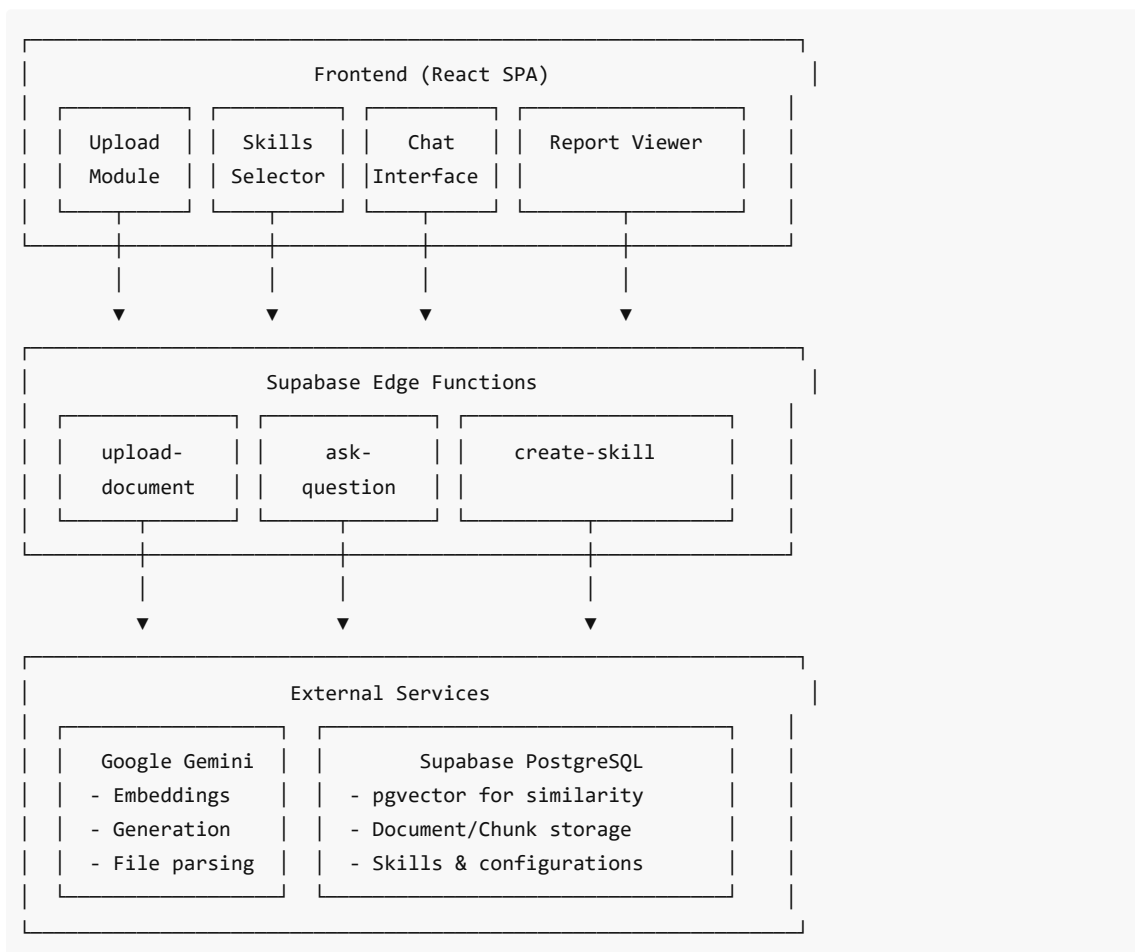
- Retrieves relevant context using vector similarity
- Generates accurate, cited answers using AI
- Applies domain expertise through configurable skills
- Produces professional-grade analysis reports

2.3 Target Users

- Financial analysts and auditors
- Compliance and risk management professionals
- Legal researchers
- Technical documentation teams
- Knowledge workers processing document-heavy workflows

3. Architecture

3.1 High-Level Architecture

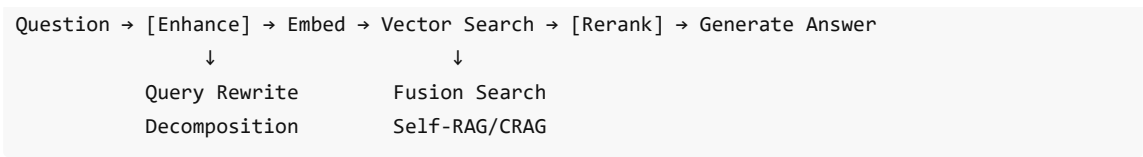


3.2 Data Flow

Document Upload Flow:

File → Parse (Gemini/LlamaParse) → Chunk → Embed → Store in pgvector

Query Flow:



3.3 Component Interaction

Component	Responsibility	Dependencies
Frontend	UI, state management, user interaction	React, shadcn-ui
Edge Functions	Business logic, AI orchestration	Gemini API, Supabase
PostgreSQL	Data persistence, vector search	pgvector extension
Gemini API	Text generation, embeddings	Google Cloud

4. Core Features

4.1 Document Management

Feature	Description
Multi-format upload	PDF, DOCX, TXT, JSON support
Processing status	Real-time status tracking
Reprocessing	Re-chunk with different configurations
Batch upload	Multiple documents simultaneously
Original text preservation	Stored for reprocessing

4.2 Intelligent Chunking

Strategy	Description	Best For
Fixed	Character-based with overlap	General use, fast processing
Semantic	AI-detected natural boundaries	Narrative documents
Proposition	Atomic fact extraction	Fact-dense technical docs
Hierarchical	Parent-child relationships	Long documents, reports

4.3 RAG Enhancements

Technique	Description	Cost
HyDE	Generate hypothetical answer for better retrieval	+1 API call
Query Rewrite	Improve vague or ambiguous questions	+1 API call
Decomposition	Break complex questions into sub-queries	+1 API call

Verification	Check answer accuracy against sources	+ 1 API call
Confidence	Score answer reliability and coverage	+ 1 API call
Reasoning	Chain-of-thought analysis	Included

4.4 Advanced Retrieval

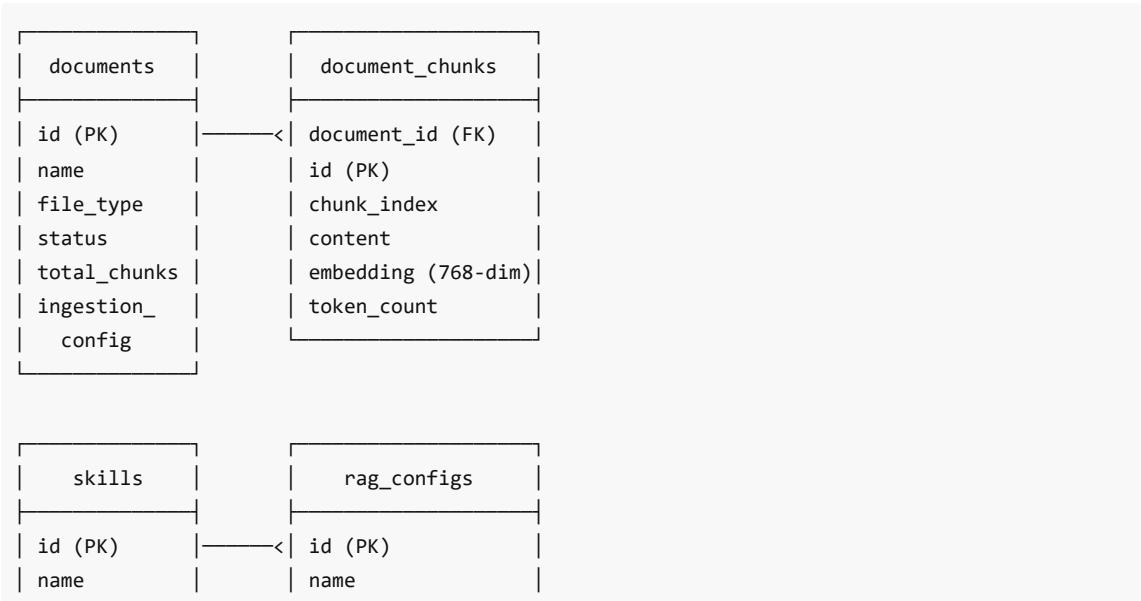
Technique	Description
Fusion Search	Combine semantic + keyword (BM25) search
Reranking	Cross-encoder or LLM-based relevance scoring
Self-RAG	Iterative retrieval with reflection
CRAG	Corrective RAG for low-relevance detection
Hierarchical Expansion	Include parent chunks for context

4.5 Output Generation

Format	Description
Narrative	Flowing prose style
Structured	Headers and bullet points
Tabular	Emphasis on tables and data
Audit	Professional audit-quality format

5. Database Schema

5.1 Entity Relationship Diagram



skill_type		enable_hyde	
output_format		enable_verification	
prompt_content		top_k	
rag_config_id	————	similarity_thresh	

tool_connectors
id (PK)
name
base_url
auth_type
actions (JSONB)

5.2 Table Definitions

documents

```
CREATE TABLE documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  file_type TEXT NOT NULL,
  original_file_id TEXT,
  original_extracted_text TEXT,
  total_chunks INTEGER DEFAULT 0,
  total_characters INTEGER DEFAULT 0,
  status TEXT DEFAULT 'processing', -- 'processing' | 'ready' | 'error'
  error_message TEXT,
  ingestion_config JSONB,
  last_reprocessed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

document_chunks

```
CREATE TABLE document_chunks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID REFERENCES documents(id) ON DELETE CASCADE,
  chunk_index INTEGER NOT NULL,
  content TEXT NOT NULL,
  token_count INTEGER,
  embedding VECTOR(768), -- Gemini text-embedding-004
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON document_chunks USING ivfflat (embedding vector_cosine_ops);
```

skills

```
CREATE TABLE skills (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID, -- NULL = global skill  
  name TEXT NOT NULL,  
  description TEXT,  
  category TEXT DEFAULT 'General',  
  icon TEXT DEFAULT '🧠',  
  prompt_content TEXT NOT NULL,  
  questions_template JSONB,  
  is_active BOOLEAN DEFAULT true,  
  is_default BOOLEAN DEFAULT false,  
  rag_config_id UUID REFERENCES rag_configs(id),  
  skill_type TEXT DEFAULT 'expert', -- 'expert' | 'generator' | 'meta'  
  output_format TEXT DEFAULT 'text', -- 'text' | 'markdown' | 'json'  
  parent_skill_id UUID REFERENCES skills(id),  
  tool_connector_ids UUID[] DEFAULT '{}',  
  created_at TIMESTAMPTZ DEFAULT now(),  
  updated_at TIMESTAMPTZ DEFAULT now()  
);
```

rag_configs

```
CREATE TABLE rag_configs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name TEXT NOT NULL,  
  description TEXT,  
  enable_hyde BOOLEAN DEFAULT false,  
  enable_query_rewrite BOOLEAN DEFAULT false,  
  enable_decomposition BOOLEAN DEFAULT false,  
  enable_verification BOOLEAN DEFAULT false,  
  enable_confidence BOOLEAN DEFAULT false,  
  enable_reasoning BOOLEAN DEFAULT false,  
  top_k INTEGER DEFAULT 15,  
  similarity_threshold FLOAT DEFAULT 0.3,  
  is_preset BOOLEAN DEFAULT false,  
  preset_category TEXT,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

tool_connectors

```
CREATE TABLE tool_connectors (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID,  
  name TEXT NOT NULL,  
  description TEXT,  
  icon TEXT DEFAULT '🔗',
```

```

connector_type TEXT DEFAULT 'http',
base_url TEXT NOT NULL,
auth_type TEXT DEFAULT 'none', -- 'none' | 'api_key' | 'bearer' | 'basic'
auth_header_name TEXT DEFAULT 'Authorization',
auth_value TEXT,
default_headers JSONB DEFAULT '{}',
timeout_ms INTEGER DEFAULT 30000,
actions JSONB DEFAULT '[]',
is_active BOOLEAN DEFAULT true,
last_tested_at TIMESTAMPTZ,
last_error TEXT,
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

```

5.3 Custom Functions

Vector Similarity Search

```

CREATE FUNCTION match_document_chunks(
    query_embedding TEXT,
    match_threshold FLOAT DEFAULT 0.3,
    match_count INT DEFAULT 15,
    filter_document_ids UUID[] DEFAULT NULL
)
RETURNS TABLE (
    id UUID,
    document_id UUID,
    document_name TEXT,
    chunk_index INT,
    content TEXT,
    similarity FLOAT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        dc.id,
        dc.document_id,
        d.name AS document_name,
        dc.chunk_index,
        dc.content,
        1 - (dc.embedding <=> query_embedding::vector) AS similarity
    FROM document_chunks dc
    JOIN documents d ON d.id = dc.document_id
    WHERE d.status = 'ready'
        AND (filter_document_ids IS NULL OR dc.document_id = ANY(filter_document_ids))
        AND 1 - (dc.embedding <=> query_embedding::vector) > match_threshold
    ORDER BY dc.embedding <=> query_embedding::vector
    LIMIT match_count;

```

```
END;  
$$;
```

6. API & Edge Functions

6.1 upload-document

Purpose: Process and index uploaded documents

Endpoint: POST /functions/v1/upload-document

Request:

```
{  
  file: File,  
  ingestionConfig: {  
    chunkingStrategy: 'fixed' | 'semantic' | 'proposition' | 'hierarchical',  
    chunkSize: number,  
    chunkOverlap: number,  
    enableContextEnrichment: boolean,  
    enableMetadataExtraction: boolean,  
    enableSummaryChunks: boolean,  
    enableEntityExtraction: boolean,  
    preserveTablesLists: boolean,  
    parserPreference: 'auto' | 'llamaparse' | 'gemini'  
  }  
}
```

Response:

```
{  
  success: boolean,  
  documentId: string,  
  totalChunks: number,  
  totalCharacters: number,  
  processingTimeMs: number  
}
```

Processing Pipeline:

1. Parse file (LlamaParse or Gemini)
2. Extract and clean text
3. Apply chunking strategy
4. Generate embeddings (batch)
5. Store chunks with vectors

6.2 ask-question

Purpose: Answer questions using RAG

Endpoint: POST /functions/v1/ask-question

Request:

```
{
  question: string,
  documentIds: string[],
  skillId?: string,
  skillType?: 'expert' | 'generator' | 'meta',
  skillName?: string,
  skillOutputFormat?: 'text' | 'markdown' | 'json',
  customPrompt?: string,
  ragConfig: {
    enableHyde: boolean,
    enableQueryRewrite: boolean,
    enableDecomposition: boolean,
    enableVerification: boolean,
    enableConfidence: boolean,
    enableReasoning: boolean,
    topK: number,
    similarityThreshold: number
  },
  retrievalConfig: {
    enableFullDocumentMode: boolean,
    fullDocumentMaxSize: number,
    enableReranking: boolean,
    rerankingMethod: 'cross-encoder' | 'llm',
    enableFusion: boolean,
    fusionStrategy: 'rrf' | 'weighted' | 'linear',
    enableSelfRag: boolean,
    selfRagMaxIterations: number,
    enableCrag: boolean,
    cragRelevanceThreshold: number
  },
  outputFormat: {
    style: 'narrative' | 'structured' | 'tabular' | 'audit',
    includeExecutiveSummary: boolean,
    citationFormat: 'inline' | 'detailed' | 'footnote',
    detailLevel: 'concise' | 'standard' | 'comprehensive'
  }
}
```

Response:

```
{
  answer: string,
  reportHtml?: string,
  reportData?: object,
  sources: Array<{
    documentName: string,
    chunkIndex: number,
    similarity: number,
    preview: string
  }>
}
```

```

    }>,
    metadata?: {
      skillUsed: string,
      ragSkillsApplied: string[],
      retrievalTechniques: string[],
      confidence?: number,
      verified?: boolean
    }
  }
}

```

6.3 create-skill

Purpose: AI-powered skill generation

Endpoint: POST /functions/v1/create-skill

Request:

```

{
  description: string, // Natural language description
  saveSkill: boolean   // Whether to persist immediately
}

```

Response:

```

{
  success: boolean,
  skill: {
    name: string,
    description: string,
    category: string,
    icon: string,
    skill_type: 'expert' | 'generator' | 'meta',
    output_format: 'text' | 'markdown' | 'json',
    prompt_content: string
  },
  reasoning: string,
  suggested_use_cases: string[],
  savedSkillId?: string
}

```

6.4 reprocess-document

Purpose: Re-chunk existing document with new configuration

Endpoint: POST /functions/v1/reprocess-document

Request:

```

{
  documentId: string,
}

```

```
  ingestionConfig: IngestionConfig
}
```

6.5 parse-prompt-document

Purpose: Extract text from uploaded prompt files

Endpoint: POST /functions/v1/parse-prompt-document

Request:

```
{
  file: File // TXT, JSON, PDF, or DOCX
}
```

7. Configuration System

7.1 Ingestion Configuration Presets

Preset	Strategy	Chunk Size	Overlap	Enhancements
Fast	Fixed	2000	200	None
Balanced	Fixed	1500	150	Context + Metadata
Accurate	Semantic	1000	100	All enabled
Financial	Proposition	800	80	Entity extraction
Legal	Semantic	1200	120	Structure preservation

7.2 RAG Configuration Presets

Preset	HyDE	Rewrite	Decomp	Verify	Confidence	Reasoning
Default	✗	✗	✗	✗	✗	✗
Enhanced	✓	✓	✓	✓	✓	✗
Accurate	✓	✓	✓	✓	✓	✓

7.3 Retrieval Configuration Presets

Preset	Full Doc	Rerank	Fusion	Self-RAG	CRAG
Fast	✗	✗	✗	✗	✗
Balanced	✗	✓	✓	✗	✗
Accurate	✗	✓	✓	✓	✓
Full Document	✓	✗	✗	✗	✗

7.4 Output Format Presets

Preset	Style	Summary	Citations	Detail
Simple	Narrative	✗	Inline	Concise
Standard	Structured	✓	Detailed	Standard
Detailed	Structured	✓	Detailed	Comprehensive
Audit	Audit	✓	Footnote	Comprehensive

8. Skills & Expert System

8.1 Skill Types

Type	Description	Output
Expert	Domain-specific knowledge and analysis	Text/Markdown
Generator	Structured document generation	JSON/Markdown
Meta	System-level operations (e.g., skill creation)	JSON

8.2 Skill Categories

- Financial Regulation
- Risk Management
- Compliance
- Audit
- Legal
- Technical
- Operations
- Document Generation
- Research
- Meta
- Custom

8.3 Pre-built Generator Skills

Skill	Description	Output Format
Skill Creator	AI-powered skill generation	JSON
SOP Creator	Standard Operating Procedure generation	Markdown
Brand Voice Generator	Brand voice and style guide extraction	Markdown
PPTX Generator	PowerPoint presentation outlines	JSON
Script Writer	Video/presentation/meeting scripts	Markdown
Reference Manager	Citation extraction and formatting	Markdown

8.4 Skill Prompt Structure

```
You are an expert in [DOMAIN]. Your expertise includes:

## Core Competencies:
- [Area 1]: [Description]
- [Area 2]: [Description]

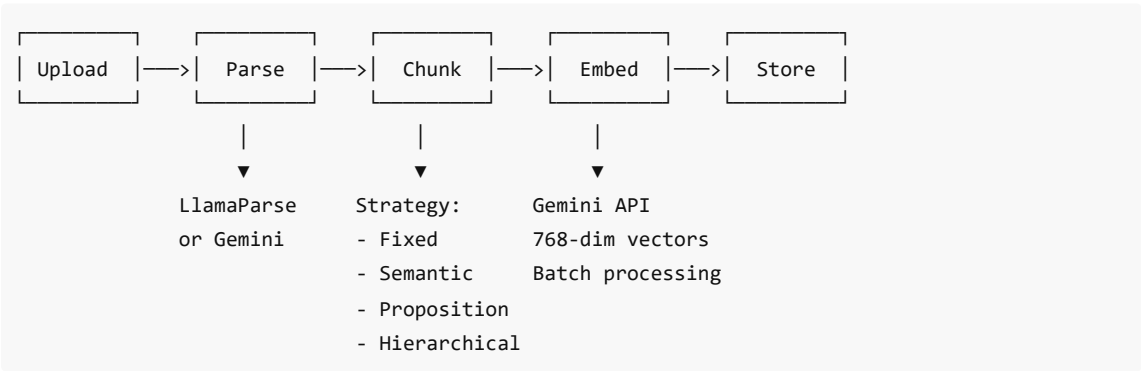
## Assessment Framework:
When analyzing documents, evaluate:
1. [Criterion 1]
2. [Criterion 2]
3. [Criterion 3]

## Response Guidelines:
- [Style guideline 1]
- [Style guideline 2]
- Always cite specific sections from documents
- Provide actionable insights and recommendations

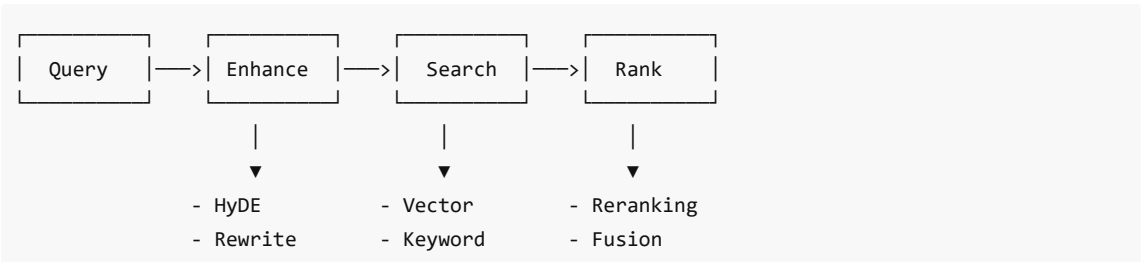
## Key Considerations:
- [Important factor 1]
- [Important factor 2]
- [Relevant standards/regulations]
```

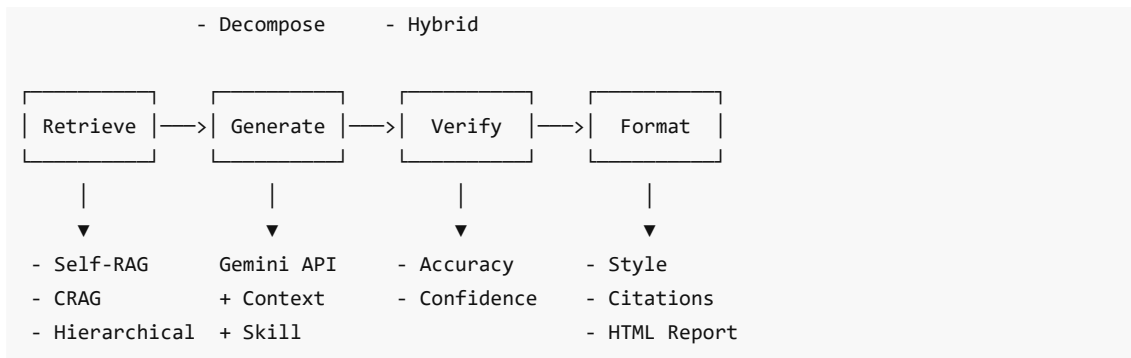
9. RAG Pipeline

9.1 Document Ingestion Pipeline



9.2 Query Processing Pipeline





9.3 Embedding Strategy

- **Model:** Gemini text-embedding-004
- **Dimensions:** 768
- **Batch Size:** Up to 100 texts per request
- **Similarity:** Cosine distance via pgvector

9.4 Retrieval Strategies

Vector Search (Default)

```

SELECT *, 1 - (embedding <=> query_embedding) AS similarity
FROM document_chunks
WHERE similarity > threshold
ORDER BY embedding <=> query_embedding
LIMIT top_k;
  
```

Fusion Search (Hybrid)

- Semantic: Vector similarity
- Keyword: BM25 text matching
- Combination: RRF, Weighted, or Linear fusion

Self-RAG (Iterative)

1. Initial retrieval
2. Generate preliminary answer
3. Reflect on quality
4. Additional retrieval if needed
5. Iterate until confident or max iterations

CRAG (Corrective)

1. Retrieve chunks
2. Score relevance
3. If low relevance: refine query or expand search
4. Generate with verified context

10. User Interface

10.1 Main Workflow



10.2 Component Hierarchy

```
Index.tsx
├─ FileUpload
├─ DocumentList
├─ SkillSelector
│   └─ SkillManager (dialog)
│       └─ SkillCreatorDialog
├─ RagConfigPanel
├─ RetrievalConfigPanel
├─ OutputFormatPanel
├─ ChatInterface
│   └─ ChatExpertSelector
│   └─ ChunkSources
│   └─ RagMetadataBadges
│       └─ GeneratorOutput
├─ ReportViewer
└─ ReprocessDialog
```

10.3 Key UI Components

Component	Purpose
FileUpload	Drag-and-drop file upload
DocumentList	Display uploaded documents with status
SkillSelector	Dropdown for expert selection
SkillManager	CRUD operations for skills

SkillCreatorDialog	AI skill generation wizard
ChatInterface	Main Q&A interaction
ChatExpertSelector	Per-message expert switching
ChunkSources	Citation display
ReportViewer	HTML report display
GeneratorOutput	Structured output renderer
RagConfigPanel	RAG skill toggles
RetrievalConfigPanel	Advanced retrieval settings
OutputFormatPanel	Output style configuration

11. Technology Stack

11.1 Frontend

Technology	Version	Purpose
React	18.3.1	UI framework
TypeScript	5.8.3	Type safety
Vite	5.4.19	Build tool
React Router	6.30.1	Routing
TanStack Query	5.83.0	Server state
React Hook Form	7.61.1	Form handling
Zod	3.25.76	Validation

11.2 UI Components

Technology	Purpose
shadcn-ui	Component library
Radix UI	Accessible primitives
Tailwind CSS	Styling
Lucide React	Icons
Recharts	Charts
Sonner	Notifications

11.3 Backend

Technology	Purpose
Supabase	BaaS platform
PostgreSQL	Database
pgvector	Vector similarity
Deno	Edge function runtime
Google Gemini	AI capabilities

11.4 External Services

Service	Purpose
Google Gemini API	Text generation, embeddings, file parsing
LlamaParse (optional)	Advanced document parsing

12. Deployment

12.1 Environment Variables

```
# Frontend (Vite)
VITE_SUPABASE_PROJECT_ID=your-project-id
VITE_SUPABASE_PUBLISHABLE_KEY=your-anon-key
VITE_SUPABASE_URL=https://your-project.supabase.co

# Edge Functions (Supabase Secrets)
GOOGLE_API_KEY=your-gemini-api-key
LLAMAPARSE_API_KEY=your-llamaparse-key # Optional
```

12.2 Database Setup

1. Create Supabase project
2. Enable pgvector extension
3. Run migration scripts:
 - 20260127000000_add_skill_types.sql
 - 20260127100000_seed_generator_skills.sql

12.3 Edge Function Deployment

```
# Deploy all functions
supabase functions deploy upload-document
supabase functions deploy ask-question
supabase functions deploy create-skill
supabase functions deploy reprocess-document
supabase functions deploy parse-prompt-document
```

12.4 Platform Integration

The project is integrated with Lovable platform:

- **Project URL:** `https://lovable.dev/projects/13af4cb5-9f84-4979-8adc-a9ad76a849ff`
- **Auto-deployment:** Git sync enabled

13. Future Roadmap

13.1 Phase 1: Tool Connectors (In Progress)

- ☐ Tool Connector Manager UI
- ☐ External API integration framework
- ☐ Action execution engine
- ☐ Skill-to-connector linking

13.2 Phase 2: Enhanced Collaboration

- ☐ User authentication
- ☐ Shared workspaces
- ☐ Document permissions
- ☐ Collaboration features

13.3 Phase 3: Advanced Analytics

- ☐ Usage analytics dashboard
- ☐ Query performance metrics
- ☐ Skill effectiveness tracking
- ☐ Cost optimization insights

13.4 Phase 4: Enterprise Features

- ☐ SSO integration
- ☐ Audit logging
- ☐ Data retention policies
- ☐ Custom model endpoints

Appendix A: API Cost Estimation

Operation	API Calls	Estimated Cost
Document upload (1MB PDF)	2-5	\$0.01-0.05
Basic question	2	\$0.01-0.02
Enhanced question (all skills)	7-8	\$0.05-0.10
Skill generation	1-2	\$0.02-0.05

Appendix B: Performance Benchmarks

Metric	Target	Typical
Document processing	< 30s/MB	10-20s/MB
Query response	< 5s	2-4s
Vector search	< 100ms	20-50ms
Skill generation	< 10s	5-8s

Document Version History:

Version	Date	Author	Changes
1.0	2025-12	System	Initial release
2.0	2026-01	System	Second Brain features, Generator Skills